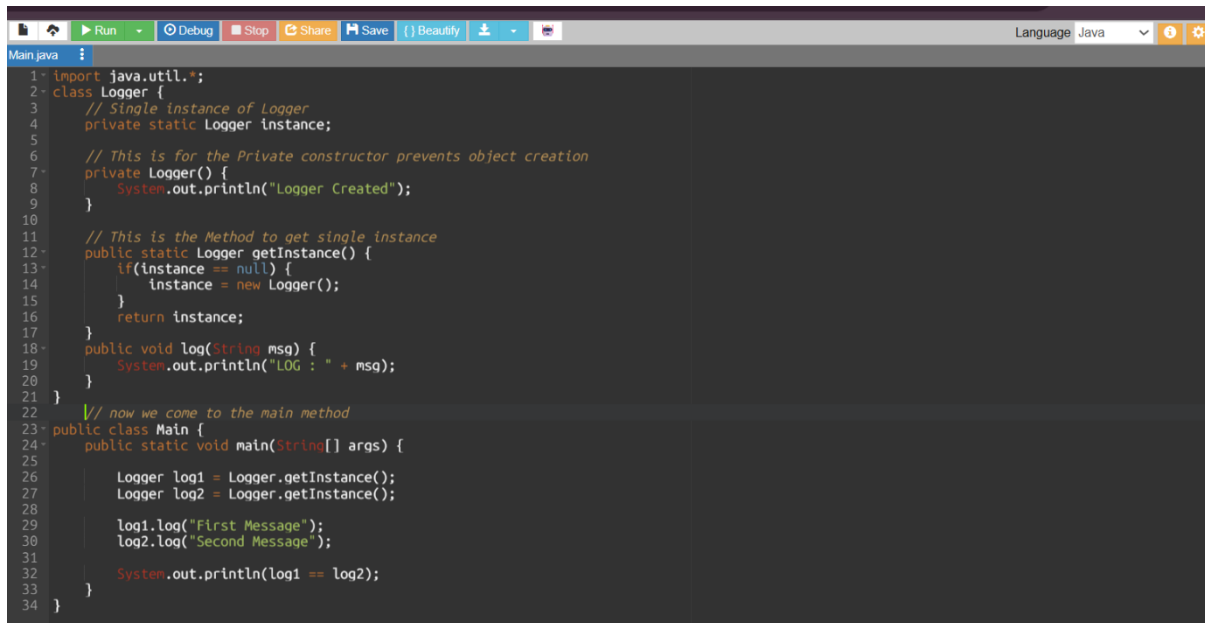


COGNIZANT DIGITAL NURTURE PROGRAM WEEK 1 HANDS ON TASK

EXERCISE 1 :

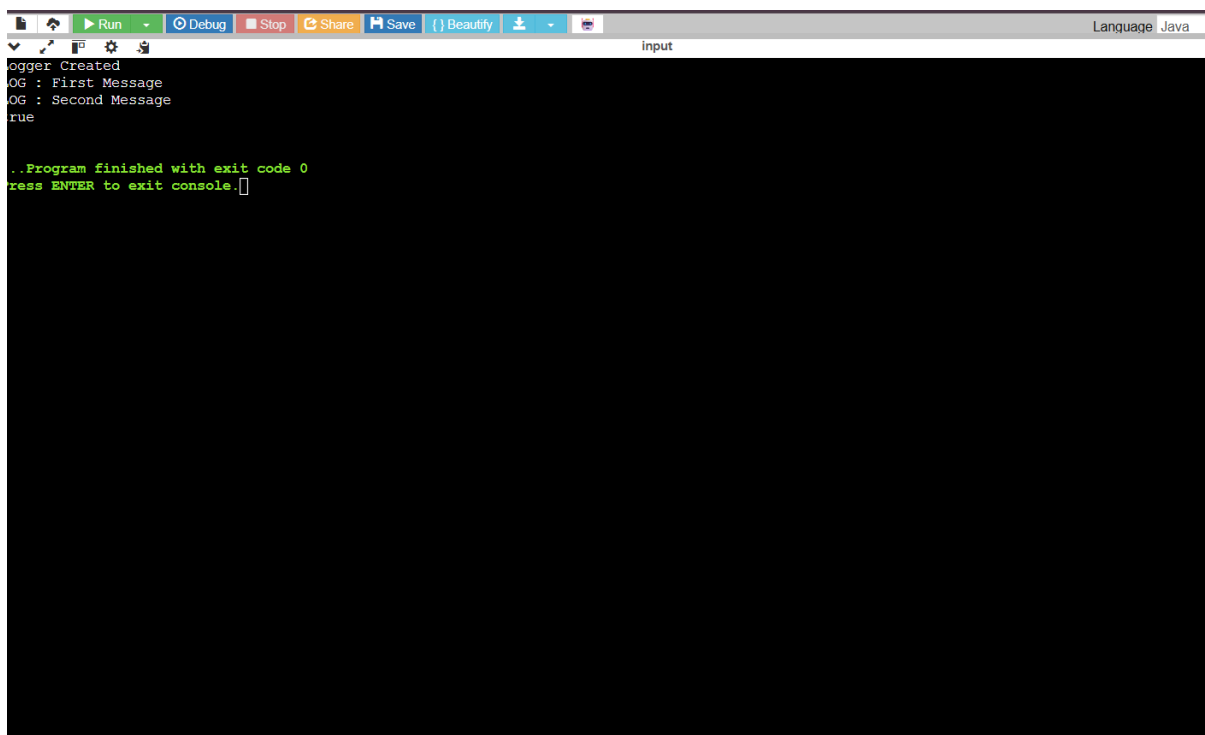
Singleton Pattern Example :

Code :



```
1- import java.util.*;
2- class Logger {
3-     // Single instance of Logger
4-     private static Logger instance;
5-
6-     // This is for the Private constructor prevents object creation
7-     private Logger() {
8-         System.out.println("Logger Created");
9-     }
10-
11-     // This is the Method to get single instance
12-     public static Logger getInstance() {
13-         if(instance == null) {
14-             instance = new Logger();
15-         }
16-         return instance;
17-     }
18-     public void log(String msg) {
19-         System.out.println("LOG : " + msg);
20-     }
21- }
22- // now we come to the main method
23- public class Main {
24-     public static void main(String[] args) {
25-
26-         Logger log1 = Logger.getInstance();
27-         Logger log2 = Logger.getInstance();
28-
29-         log1.log("First Message");
30-         log2.log("Second Message");
31-
32-         System.out.println(log1 == log2);
33-     }
34- }
```

Output :



```
logger Created
LOG : First Message
LOG : Second Message
true

..Program finished with exit code 0
Press ENTER to exit console.
```

EXERCISE 2 :

Factory Pattern Example :

Code :

```
1  import java.util.*;
2  interface Document {
3      void open();
4  }
5  // Here We Created a Class For Word Document
6  class WordDocument implements Document {
7      public void open() {
8          System.out.println("The Word Document is Opened");
9      }
10 }
11 // Here We Created a Class For PDF Document
12 class PdfDocument implements Document {
13     public void open() {
14         System.out.println("The PDF Document is Opened");
15     }
16 }
17 // Here We Created a Class For Excel Document
18 class ExcelDocument implements Document {
19     public void open() {
20         System.out.println("The Excel Document is Opened");
21     }
22 }
23 abstract class DocumentFactory {
24     abstract Document createDocument();
25 }
26
27 class WordFactory extends DocumentFactory {
28     Document createDocument() {
29         return new WordDocument();
30     }
31 }
32
33 class PdfFactory extends DocumentFactory {
34     Document createDocument() {
35         return new PdfDocument();
36     }
37 }
38 // Now we come to the Main Method
39 public class Main {
40     public static void main(String[] args) {
41
42         DocumentFactory factory = new WordFactory();
43         Document doc = factory.createDocument();
44         doc.open();
45     }
46 }
47
48 }
```

Output :

```
the Word Document is Opened

..Program finished with exit code 0
Press ENTER to exit console.
```

Exercise 3 :

Builder Pattern Example :

Code :

```
1- import java.util.*;
2- // Here we created a class for Computer
3- class Computer {
4-     //Here we Taken Input for CPU,Ram and Storage
5-     private String cpu;
6-     private int ram;
7-     private int storage;
8-     private Computer(Builder builder) {
9-         this.cpu = builder.cpu;
10-        this.ram = builder.ram;
11-        this.storage = builder.storage;
12-    }
13-    public void show() {
14-        System.out.println(cpu + " " + ram + "GB " + storage + "GB");
15-    }
16-    // Here We created a Class For Builder and taken input for same cpu , ram and storage
17-    static class Builder {
18-        private String cpu;
19-        private int ram;
20-        private int storage;
21-        public Builder setCPU(String cpu) {
22-            this.cpu = cpu;
23-            return this;
24-        }
25-        public Builder setRAM(int ram) {
26-            this.ram = ram;
27-            return this;
28-        }
29-        public Builder setStorage(int storage) {
30-            this.storage = storage;
31-            return this;
32-        }
33-        public Computer build() {
34-            return new Computer(this);
35-        }
36-    }
37-}
38- // Now we come to the Main Method
39- public class Main {
40-     public static void main(String[] args) {
41-         Computer c = new Computer.Builder()
42-             .setCPU("i7")
43-             .setRAM(16)
44-             .setStorage(512)
45-             .build();
46-         c.show();
47-     }
48- }
```

Output :

```
i7 16GB 512GB
...Program finished with exit code 0
Press ENTER to exit console.
```

Exercise 4 :

Adapter Pattern Example :

Code :

```
1 import java.util.*;
2
3 // Here We Create an Interface Named PaymentProcessor
4 interface PaymentProcessor {
5     void processPayment(double amount);
6 }
7
8 // Created a Class for PayPalGateway
9 class PayPalGateway {
10     void makePayment(double amount) {
11         System.out.println("PayPal Paid : " + amount);
12     }
13 }
14
15 // Now we Create an Adapter Class For each Payment
16 class PayPalAdapter implements PaymentProcessor {
17     private PayPalGateway gateway;
18
19     public PayPalAdapter() {
20         gateway = new PayPalGateway();
21     }
22
23     public void processPayment(double amount) {
24         gateway.makePayment(amount);
25     }
26 }
27
28 // Now we come to the main method
29 public class Main {
30     public static void main(String[] args) {
31
32         PaymentProcessor payment =
33             new PayPalAdapter();
34
35         payment.processPayment(5000);
36     }
37 }
```

Output :

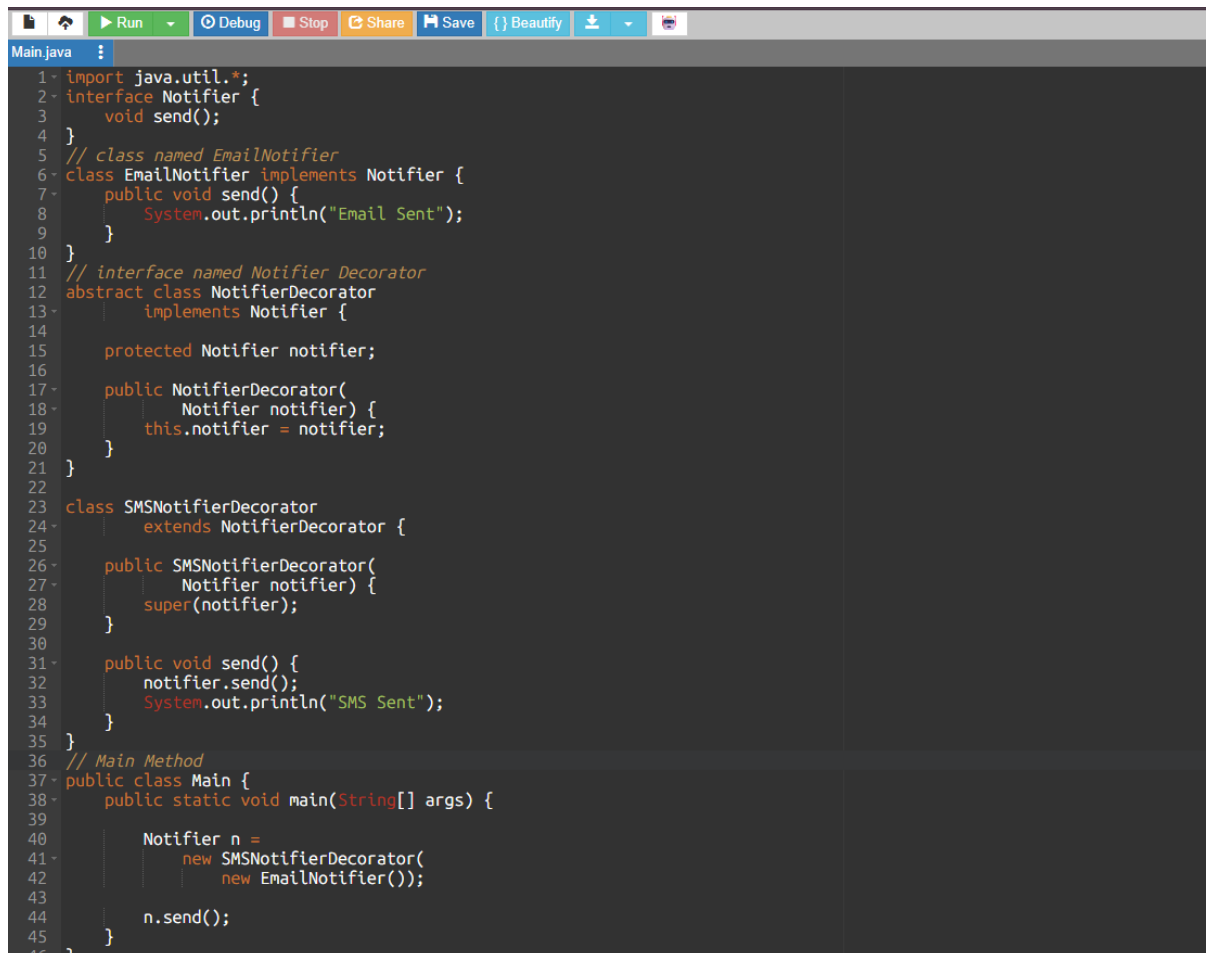
```
PayPal Paid : 5000.0

...Program finished with exit code 0
Press ENTER to exit console.
```

Exercise 5 :

DecoratorPatternExample :

Code :



```
1: import java.util.*;
2: interface Notifier {
3:     void send();
4: }
5: // class named EmailNotifier
6: class EmailNotifier implements Notifier {
7:     public void send() {
8:         System.out.println("Email Sent");
9:     }
10: }
11: // interface named Notifier Decorator
12: abstract class NotifierDecorator
13:     implements Notifier {
14:
15:     protected Notifier notifier;
16:
17:     public NotifierDecorator(
18:         Notifier notifier) {
19:         this.notifier = notifier;
20:     }
21: }
22:
23: class SMSNotifierDecorator
24:     extends NotifierDecorator {
25:
26:     public SMSNotifierDecorator(
27:         Notifier notifier) {
28:         super(notifier);
29:     }
30:
31:     public void send() {
32:         notifier.send();
33:         System.out.println("SMS Sent");
34:     }
35: }
36: // Main Method
37: public class Main {
38:     public static void main(String[] args) {
39:
40:         Notifier n =
41:             new SMSNotifierDecorator(
42:                 new EmailNotifier());
43:
44:         n.send();
45:     }
46: }
```

Output :



```
Email Sent
SMS Sent

...Program finished with exit code 0
Press ENTER to exit console.
```

Example 6 :

Proxy Pattern Example :

Code :

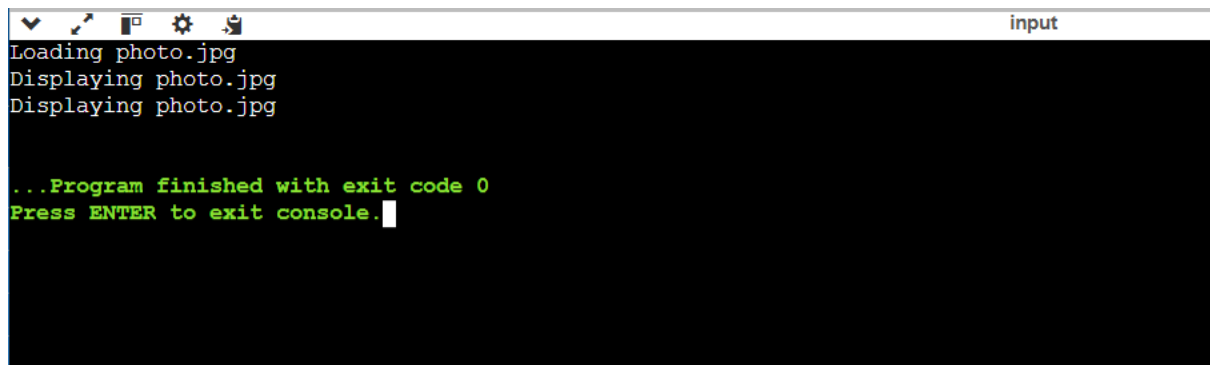
```
1- import java.util.*;
2- interface Image {
3-     void display();
4- }
5- //class named RealImage
6- class RealImage implements Image {
7-
8-     private String fileName;
9-
10-     public RealImage(String fileName) {
11-         this.fileName = fileName;
12-         loadImage();
13-     }
14-     //loadimage
15-     private void loadImage() {
16-         System.out.println("Loading " + fileName);
17-     }
18-     //Display
19-     public void display() {
20-         System.out.println("Displaying " + fileName);
21-     }
22- }
23- // class named proxyimage
24- class ProxyImage implements Image {
25-
26-     private RealImage realImage;
27-     private String fileName;
28-
29-     public ProxyImage(String fileName) {
30-         this.fileName = fileName;
31-     }
32-
33-     public void display() {
34-
35-         if(realImage == null) {
36-             realImage =
37-                 new RealImage(fileName);
38-         }
39-
40-         realImage.display();
41-     }
42- }
43- // Main Method-
44- public class Main {
45-     public static void main(String[] args) {
46-
```

```
// Main Method-
public class Main {
    public static void main(String[] args) {

        Image img =
            new ProxyImage("photo.jpg");

        img.display();
        img.display();
    }
}
```

Output :

A screenshot of a terminal window with a title bar containing standard window controls and the name 'input'. The terminal has a black background with white text. The output shows three lines of status messages, followed by a green message indicating the program has finished, and a final green prompt for the user to press ENTER to exit the console.

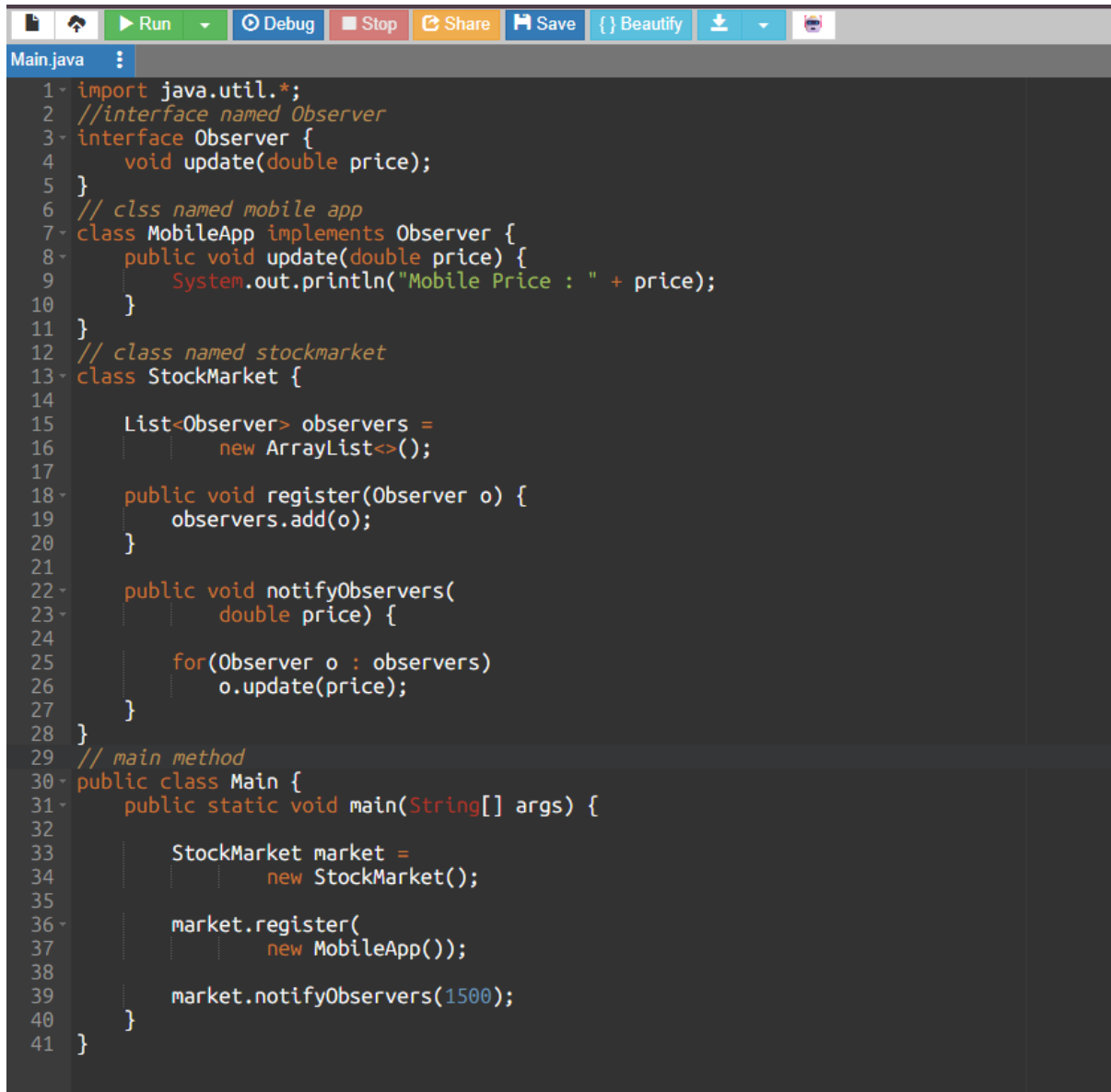
```
Loading photo.jpg
Displaying photo.jpg
Displaying photo.jpg

...Program finished with exit code 0
Press ENTER to exit console.
```

Example 7:

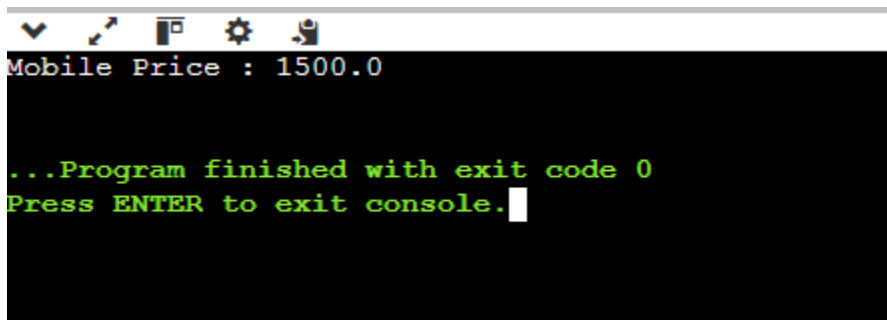
Observer Pattern Example :

Code :

A screenshot of an IDE window titled 'Main.java'. The code implements the Observer Pattern. It starts with an import for java.util.*. Then, an interface 'Observer' is defined with a 'void update(double price);' method. A class 'MobileApp' implements 'Observer' and overrides the 'update' method to print 'Mobile Price : ' followed by the price. A 'StockMarket' class is defined with a 'List<Observer> observers' (an ArrayList), a 'register(Observer o)' method to add observers, and a 'notifyObservers(double price)' method that iterates through the observers and calls their 'update' method. Finally, a 'Main' class contains a 'main' method that creates a 'StockMarket' object, registers a new 'MobileApp' object, and calls 'notifyObservers(1500)'.

```
1- import java.util.*;
2- //interface named Observer
3- interface Observer {
4-     void update(double price);
5- }
6- // class named mobile app
7- class MobileApp implements Observer {
8-     public void update(double price) {
9-         System.out.println("Mobile Price : " + price);
10-    }
11- }
12- // class named stockmarket
13- class StockMarket {
14-
15-     List<Observer> observers =
16-         new ArrayList<>();
17-
18-     public void register(Observer o) {
19-         observers.add(o);
20-     }
21-
22-     public void notifyObservers(
23-         double price) {
24-
25-         for(Observer o : observers)
26-             o.update(price);
27-     }
28- }
29- // main method
30- public class Main {
31-     public static void main(String[] args) {
32-
33-         StockMarket market =
34-             new StockMarket();
35-
36-         market.register(
37-             new MobileApp());
38-
39-         market.notifyObservers(1500);
40-     }
41- }
```

Output :

A screenshot of a console window. The first line of output is 'Mobile Price : 1500.0'. Below this, there are two lines of green text: '...Program finished with exit code 0' and 'Press ENTER to exit console.' followed by a white cursor.

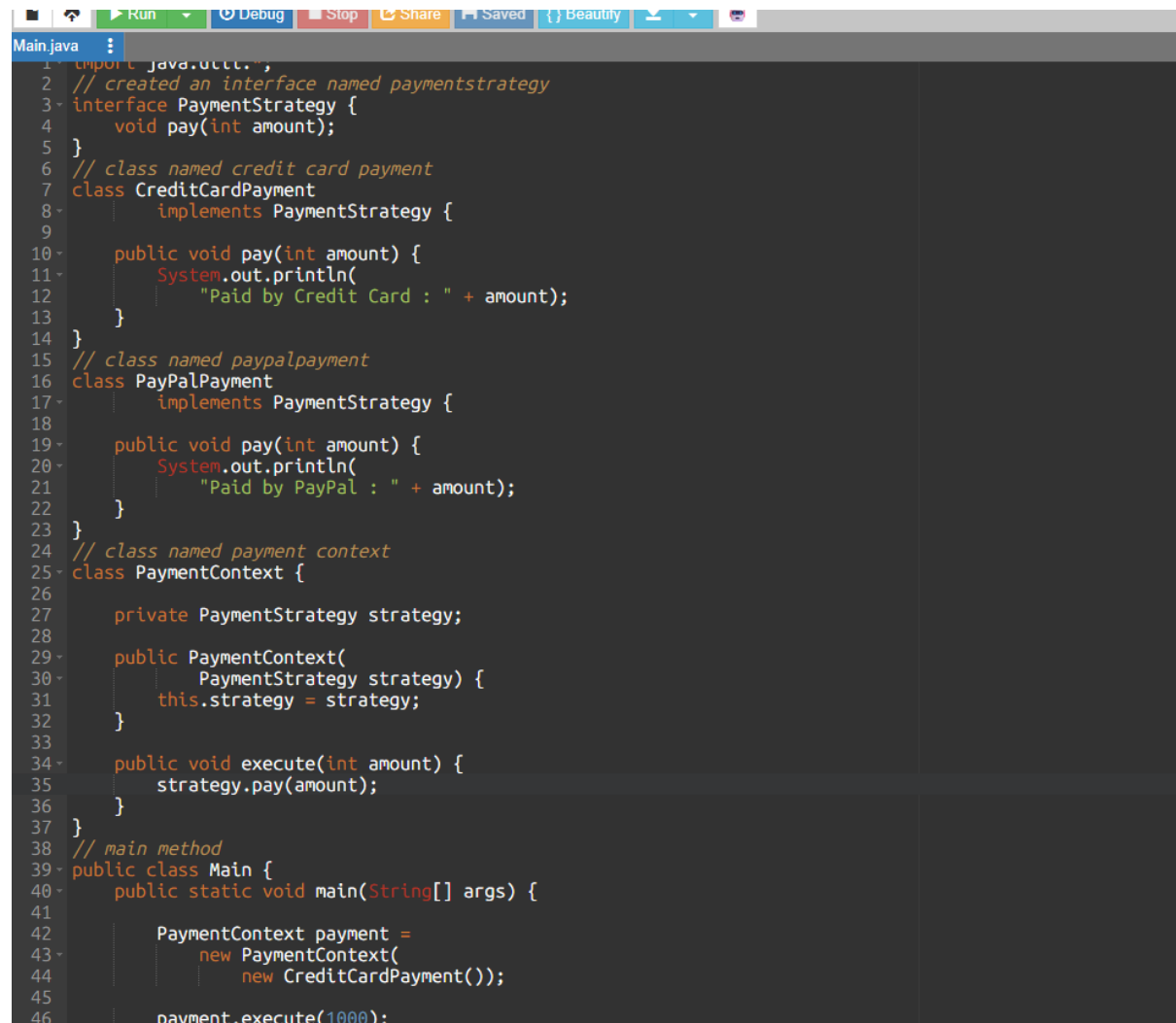
```
Mobile Price : 1500.0

...Program finished with exit code 0
Press ENTER to exit console.
```

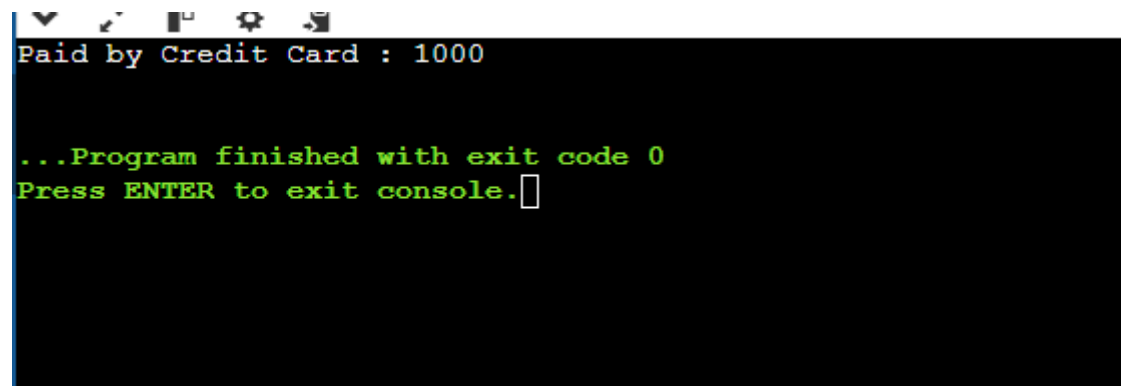

Example 8 :

Strategy Pattern Example :

Code :

A screenshot of an IDE window titled 'Main.java'. The code implements the Strategy Pattern. It starts with an import statement for java.util.*. Then, it defines an interface 'PaymentStrategy' with a 'pay(int amount)' method. Two classes, 'CreditCardPayment' and 'PayPalPayment', implement this interface. 'CreditCardPayment' prints 'Paid by Credit Card : ' followed by the amount. 'PayPalPayment' prints 'Paid by PayPal : ' followed by the amount. A 'PaymentContext' class holds a 'PaymentStrategy' object and has an 'execute(int amount)' method that calls 'pay()' on the strategy. Finally, a 'Main' class has a 'main' method that creates a 'PaymentContext' with a 'CreditCardPayment' strategy and calls 'execute(1000)'.

Output :

A screenshot of a console window. The first line of output is 'Paid by Credit Card : 1000'. The second line is '...Program finished with exit code 0'. The third line is 'Press ENTER to exit console.' followed by a cursor.

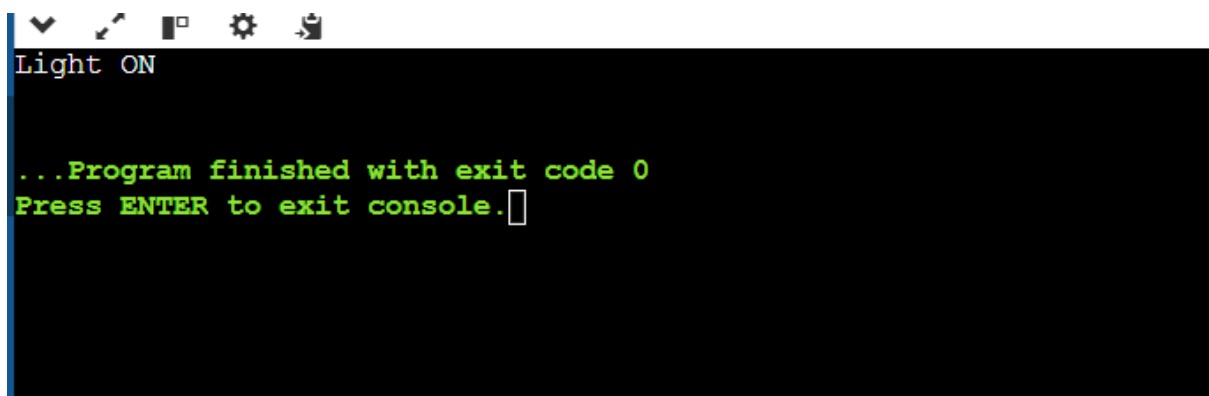
Example 9 :

Command Pattern Example :

Code :

```
1  import java.util.*;
2  // created an interface named command
3  interface Command {
4      void execute();
5  }
6  // class named light
7  class Light {
8
9      public void on() {
10         System.out.println("Light ON");
11     }
12
13     public void off() {
14         System.out.println("Light OFF");
15     }
16 }
17 // class named lightoncommand to execute command
18 class LightOnCommand
19     implements Command {
20
21     private Light light;
22
23     public LightOnCommand(
24         Light light) {
25         this.light = light;
26     }
27
28     public void execute() {
29         light.on();
30     }
31 }
32 // class named remotecontrol
33 class RemoteControl {
34
35     private Command command;
36
37     public RemoteControl(
38         Command command) {
39         this.command = command;
40     }
41
42     public void pressButton() {
43         command.execute();
44     }
45 }
46 // main method
47 public class Main {
48     public static void main(String[] args) {
49
50         Light light = new Light();
51
52         Command on =
53             new LightOnCommand(light);
54
55         RemoteControl remote =
56             new RemoteControl(on);
57
58         remote.pressButton();
59     }
60 }
```

Output :

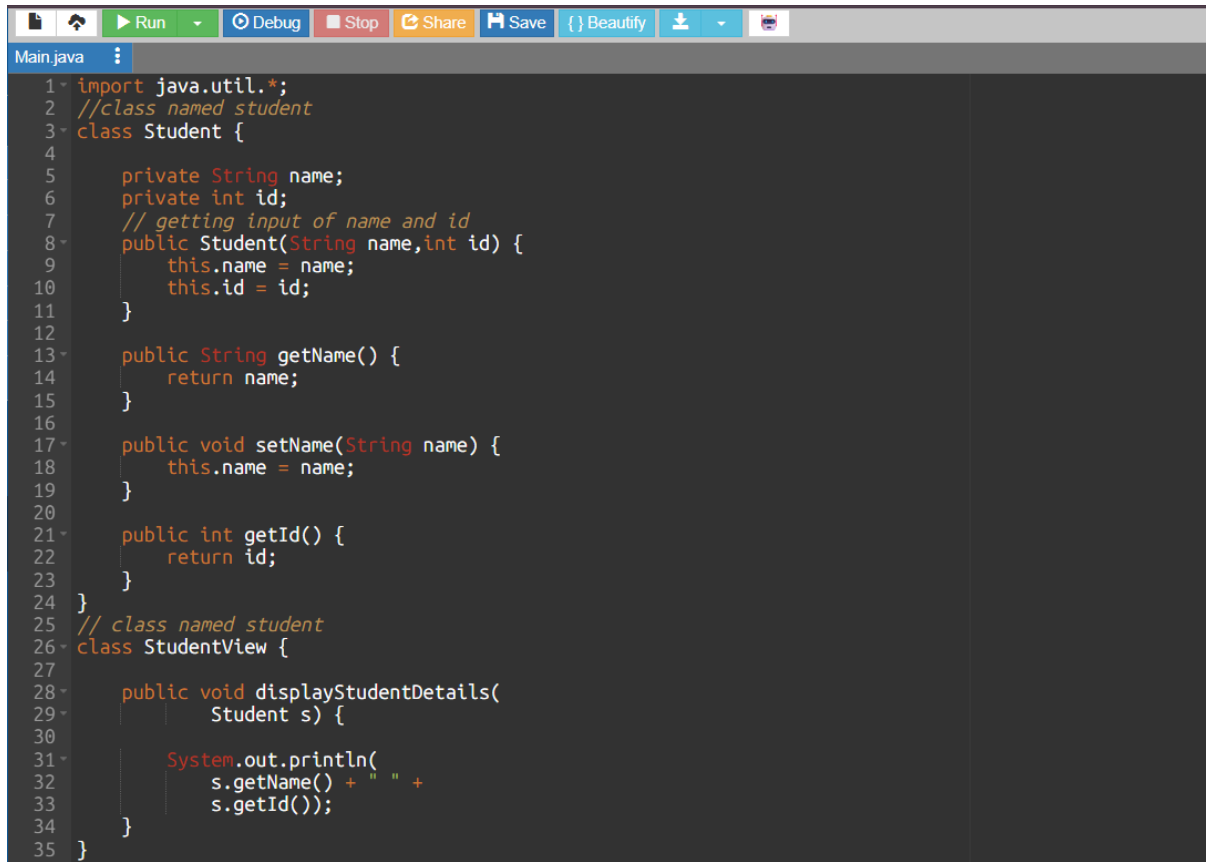
A screenshot of a terminal window with a black background. The window has a title bar with standard icons (minimize, maximize, close, settings, and a terminal icon). The text inside the terminal is as follows:

```
Light ON  
  
...Program finished with exit code 0  
Press ENTER to exit console.
```

Example 10 :

MVC Pattern Example :

Code :



```
1 import java.util.*;
2 //class named student
3 class Student {
4
5     private String name;
6     private int id;
7     // getting input of name and id
8     public Student(String name,int id) {
9         this.name = name;
10        this.id = id;
11    }
12
13    public String getName() {
14        return name;
15    }
16
17    public void setName(String name) {
18        this.name = name;
19    }
20
21    public int getId() {
22        return id;
23    }
24 }
25 // class named student
26 class StudentView {
27
28     public void displayStudentDetails(
29         Student s) {
30
31         System.out.println(
32             s.getName() + " " +
33             s.getId());
34     }
35 }
```

```

// class named student controller
class StudentController {

    private Student model;
    private StudentView view;

    public StudentController(
        Student model,
        StudentView view) {

        this.model = model;
        this.view = view;
    }

    public void updateName(
        String name) {

        model.setName(name);
    }

    public void updateView() {
        view.displayStudentDetails(model);
    }
}

// main method
public class Main {
    public static void main(String[] args) {

        Student s =
            new Student("Nitin",101);

        StudentView view =
            new StudentView();

        StudentController controller =
            new StudentController(s,view);

        controller.updateView();

        controller.updateName("Kumar");

        controller.updateView();
    }
}

```

Output :

A screenshot of a console window with a black background and white text. The window has a title bar with standard icons (minimize, maximize, close, settings, and a user icon). The output text is as follows:

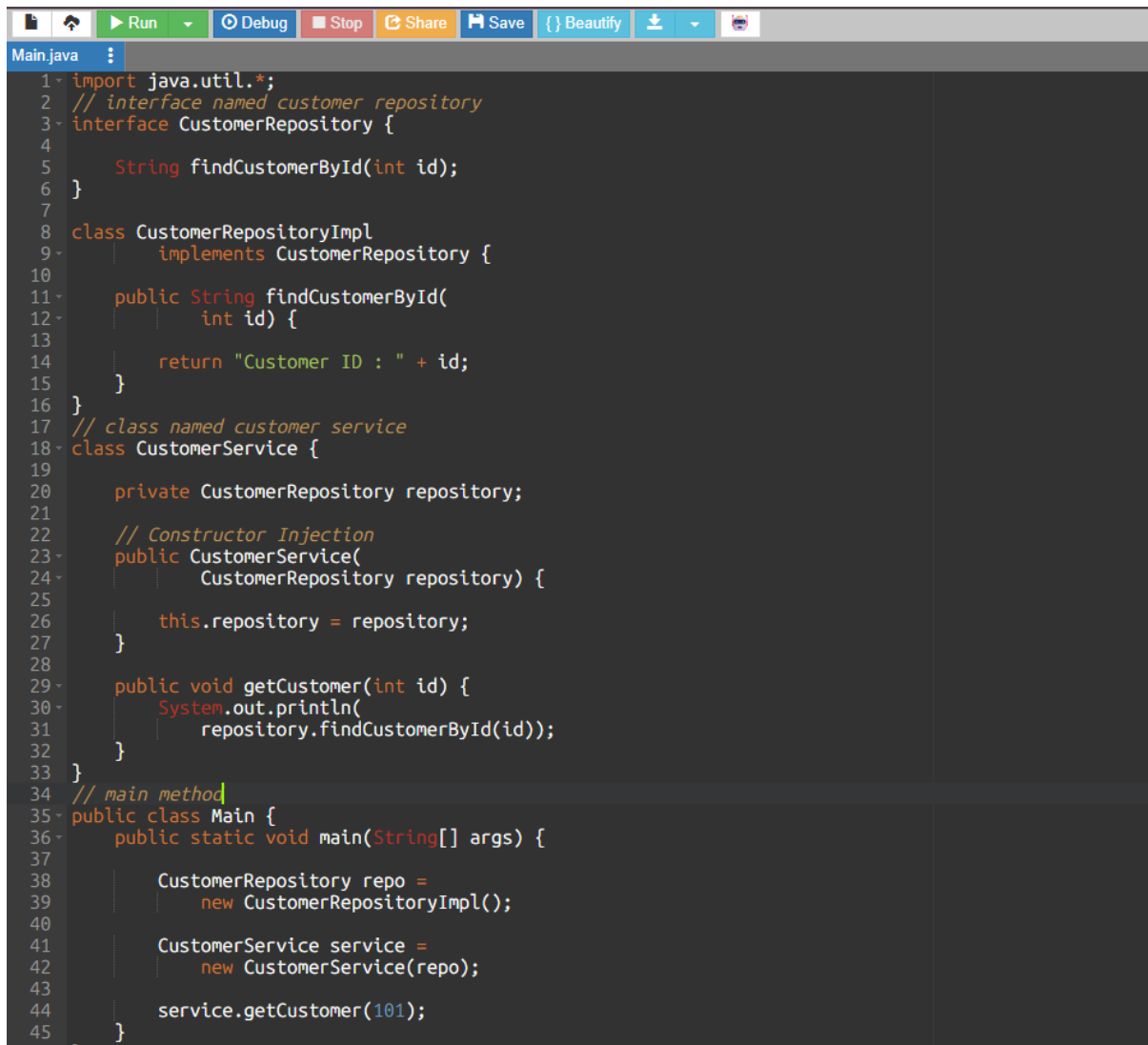
```
Nitin 101
Kumar 101

...Program finished with exit code 0
Press ENTER to exit console.
```

Example 11 :

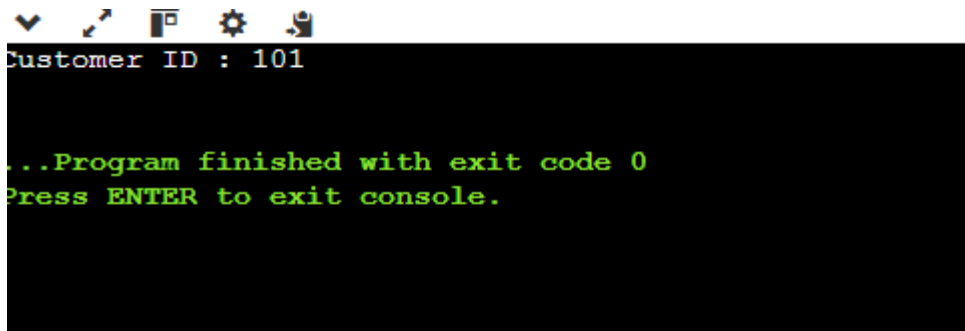
Dependency Injection Pattern Example :

Code :



```
1- import java.util.*;
2- // interface named customer repository
3- interface CustomerRepository {
4-
5-     String findCustomerById(int id);
6- }
7-
8- class CustomerRepositoryImpl
9-     implements CustomerRepository {
10-
11-     public String findCustomerById(
12-         int id) {
13-
14-         return "Customer ID : " + id;
15-     }
16- }
17- // class named customer service
18- class CustomerService {
19-
20-     private CustomerRepository repository;
21-
22-     // Constructor Injection
23-     public CustomerService(
24-         CustomerRepository repository) {
25-
26-         this.repository = repository;
27-     }
28-
29-     public void getCustomer(int id) {
30-         System.out.println(
31-             repository.findCustomerById(id));
32-     }
33- }
34- // main method
35- public class Main {
36-     public static void main(String[] args) {
37-
38-         CustomerRepository repo =
39-             new CustomerRepositoryImpl();
40-
41-         CustomerService service =
42-             new CustomerService(repo);
43-
44-         service.getCustomer(101);
45-     }
46- }
```

Output :



```
Customer ID : 101

...Program finished with exit code 0
Press ENTER to exit console.
```